

Latent Synth: A Real-Time RAVE Audio Plugin with Flow-Field Latent Control

Kejie Li

Electronic Production and Design

Berklee College of Music

nolli0117@gmail.com

Abstract—Latent Synth turns a pre-trained RAVE audio model into a real-time instrument. The project embeds a TorchScript-exported RAVE model inside both a JUCE-based VST3 plugin and a standalone application, then adds a flow-field controller that maps particle motion into an eight-dimensional latent control signal. It also embeds a p5.js visual interface inside the plugin, allowing the user to shape sound through visual motion rather than through conventional sliders alone.

This report describes the system design, the latent-control interface, the real-time plugin implementation, and the instrument’s creative use. It focuses on the control, interaction, and deployment layers that make a pre-trained RAVE model playable inside a DAW-based workflow.

Index Terms—neural audio synthesis, RAVE, VST3, JUCE, latent space, real-time audio, interactive music systems

I. INTRODUCTION

Latent Synth is a real-time audio plugin that turns a pre-trained RAVE model into a playable instrument inside a digital audio workstation. The project combines neural audio synthesis, plugin development, and visual interaction design. Its central contribution is a visual flow-field controller that turns particle motion into latent-space motion.

RAVE models can encode audio into a latent space and decode latent trajectories back into sound [1]. This makes them useful for timbre transformation, neural resynthesis, and experimental sound design. However, a trained model does not automatically provide a musical interface. A musician still needs a way to explore, control, and perform the model.

Latent Synth focuses on the gap between the model and the instrument. The plugin gives the user two ways to shape RAVE’s latent space. In native mode, the user edits latent dimensions directly with scale, bias, and jitter controls. In flow-field mode, a visual particle system generates continuous latent movement. The user shapes the behavior of the flow field, and the flow field shapes the resulting sound.

The project has three main parts:

- a RAVE sound engine running inside a JUCE plugin;
- a latent-control system with both direct and flow-field modes;
- a p5.js visual interface embedded inside the plugin.

Together, these parts turn RAVE from a pre-trained audio model into a performable sound-design tool.

II. RAVE AS THE SOUND ENGINE

RAVE is the neural audio engine in Latent Synth. A trained RAVE model analyzes audio, encodes it into a lower-dimensional latent space, and reconstructs it from that representation [1]. Latent Synth uses this learned latent space as the primary site of interaction.

In a conventional synthesizer or effects plugin, the user often controls explicit parameters such as cutoff frequency, resonance, envelope time, gain, feedback, and mix. RAVE works differently. Its latent space represents learned audio features rather than hand-designed controls. Moving through this space can change timbre, texture, density, articulation, or gesture, depending on the model and training data.

The plugin can load compatible TorchScript RAVE models and run them in real time [2]. The control system in Latent Synth is built around these exported models.

The audio path follows a simple process:

- 1) The plugin receives input audio.
- 2) RAVE encodes the audio into latent values.
- 3) Latent Synth modifies those latent values.
- 4) RAVE decodes the modified values back into audio.
- 5) The plugin outputs the result to the DAW.

The main creative decision happens in the third step. Latent Synth determines how the latent representation moves over time.

III. LATENT CONTROL DESIGN

Latent Synth offers two latent-control modes: native mode and flow-field mode.

These modes support different creative workflows. Native mode gives the user direct control over latent dimensions. Flow-field mode creates autonomous but steerable latent motion.

A. Native Latent Mode

Native mode works like a conventional plugin control surface. The user adjusts scale, bias, and jitter for latent dimensions.

Scale changes the strength of a latent value. Bias shifts the value up or down. Jitter adds controlled randomness. Together, these controls let the user shape the model in a direct and repeatable way.

This mode works well for sound design. It lets the user hear how individual latent dimensions affect the current model. It

also gives the user stable control when they want a specific timbral change rather than continuous motion.

B. Flow-Field Mode

Flow-field mode replaces direct editing with visual motion. In this mode, a particle system generates control values for the first eight latent dimensions.

The user does not automate each latent dimension by hand. Instead, the user controls the behavior of the flow field. The motion of the field becomes the motion of the latent space.

This changes the feel of the instrument. The user steers a dynamic system rather than setting fixed values. The sound moves continuously through RAVE’s latent space, and the visual interface gives the user a way to shape that movement.

IV. FLOW-FIELD LATENT CONTROLLER

The flow-field controller gives Latent Synth its main creative identity. It turns visual motion into latent-space motion.

The controller simulates particles moving through a two-dimensional noise field. At each step, it measures the state of the particle system and converts those measurements into an eight-dimensional latent control signal:

$$z_{\text{flow}} = F(P, \text{speed}, \text{noiseScale}, \text{curve}, \text{gain}, \text{contrast}, \text{intensity}), \text{controller} \quad (1)$$

where P is the particle state and z_{flow} is the latent vector used in flow-field mode.

The controller uses several aggregate features:

- average particle x -position;
- average particle y -position;
- average local noise value;
- average flow angle;
- mean horizontal velocity;
- mean vertical velocity;
- normalized particle spread;
- normalized noise variance.

These features describe the motion of the particle system as a whole. The controller does not treat one cursor position as the control source. Instead, it listens to the collective behavior of the field: where the particles gather, how fast they move, how widely they spread, and how the noise field bends their direction.

This design makes the control signal more performative. Average position gives the system a broad location in the visual field. Velocity and angle give it direction. Spread describes how dispersed the particle system has become. Noise variance describes the texture of the local field. Together, these features produce a latent trajectory that changes continuously over time.

The controller converts aggregate particle statistics into a target latent vector, then smooths the current latent vector toward that target over time. It also computes the strength of the latent movement from motion energy, spread, noise variance, and intensity. It adds low-frequency oscillation, smoothed randomness, and motion-dependent jitter. The final latent signal is smoothed over time to avoid abrupt sound changes.

The main user-facing flow controls are:

- **Speed:** changes how fast the particles move.
- **Noise scale:** changes the size and density of the flow field.
- **Curve:** changes the direction and curvature of the motion.
- **Gain:** changes how strongly the flow affects the latent space.
- **Contrast:** emphasizes direction and variation.
- **Intensity:** expands or reduces the range of latent movement.

Internally, speed and gain combine into an effective flow strength, while remaining separate in the interface as rate and depth controls. These controls let the user shape latent motion at a higher level. The user does not need to understand what each latent dimension means. The user performs the behavior of the system, and the system moves through the model.

V. VISUAL INTERFACE

Latent Synth uses p5.js for the visual interface. The p5.js library supports interactive graphics, particle systems, and rapid visual design, which makes it a good fit for the flow-field controller [3].

The interface shows the flow field and exposes its main parameters. When the user changes speed, noise scale, curve, gain, contrast, or intensity, the particles move differently. That motion changes the latent control signal. The sound changes with it.

The visual interface is part of the instrument. It does not merely show what the plugin is doing. It controls the model.

The p5.js interface runs inside the JUCE plugin through a WebBrowserComponent. During the build process, the web assets are bundled with the plugin. At runtime, the plugin loads the local web page in the editor [4].

A custom URL bridge connects the web interface to the C++ processor. The web layer sends flow parameters, latent state, model commands, and other control messages. The C++ layer parses those messages and updates the audio engine.

This design keeps the project modular. The audio engine stays in C++, where real-time performance and plugin integration matter most. The visual interface stays in p5.js, where interaction design and graphics are easier to develop.

VI. SYSTEM OVERVIEW

Latent Synth has two main software layers: a C++ audio layer and a web-based visual layer.

The C++ layer handles audio I/O, plugin parameters, model loading, buffering, RAVE inference, and plugin packaging. JUCE provides the plugin framework, including VST3 and standalone builds [5], [6]. libtorch runs the TorchScript-exported RAVE model directly in C++, without requiring a Python runtime [7].

The visual layer uses p5.js [3]. It draws the flow field, simulates particle motion, and exposes the controls that shape the latent controller. This layer does more than display animation; it produces control data that changes the sound.

A custom bridge connects the web interface to the plugin. The p5.js layer sends messages to the C++ layer. These messages update flow parameters, latent-control state, model-selection commands, and other plugin state. The C++ layer parses these messages and applies them to the audio engine.

The system has three connected roles:

- RAVE provides the learned audio representation.
- The JUCE plugin runs the model in a real-time DAW environment.
- The flow-field interface generates playable control signals for the latent space.

This structure keeps the real-time audio engine separate from the visual interface. C++ handles audio and inference. p5.js handles visual interaction. The bridge connects them.

The system can be understood as a pipeline from audio to latent space, from visual motion to latent control, and from modified latent values back to audio.

VII. REAL-TIME PLUGIN IMPLEMENTATION

The plugin receives audio in blocks from the host. RAVE, however, works on model frames that may be larger than the host buffer size. These two timing systems do not always match, so Latent Synth uses circular buffers.

Incoming samples enter an input buffer. When the buffer contains enough samples for a RAVE frame, the plugin sends that frame to the model. The model returns a decoded output frame. The plugin writes that frame into an output buffer. The host then reads from the output buffer in its own block size.

This buffering design lets the plugin run across different DAW buffer settings while still giving RAVE the frame sizes it expects.

Latent Synth also separates model inference from the immediate audio callback. TorchScript inference can take too long or behave too unpredictably to run safely on the real-time audio thread. The plugin therefore performs model processing on a compute thread and passes audio through buffers.

The plugin includes basic protection around model loading and inference. If a model fails to load, exposes an incompatible method, or throws an inference error, the processor can mute the output instead of crashing the host. This matters because user-supplied RAVE models may differ in latent dimension count, channel behavior, frame size, and export format.

VIII. MODEL MANAGEMENT

Latent Synth does not ship with pre-trained RAVE models. Instead, the user imports compatible TorchScript RAVE models into a local model directory.

The plugin includes a model explorer for model discovery, import, selection, and refresh. This lets users test different RAVE models without rebuilding the plugin.

This design keeps Latent Synth flexible. It is not one fixed synthesizer with one fixed sound. It is a control environment for different RAVE models. A user can load models trained on different source material and use the same flow-field interface to explore each model's latent space.

IX. CREATIVE USE

Latent Synth can function as both a sound-design tool and a performance instrument.

In native mode, the user edits latent values directly. This mode supports careful timbral adjustment and repeatable sound design. It helps the user learn how specific latent dimensions affect the model.

In flow-field mode, the user performs the model through motion. The particle system moves, and the sound follows. Increasing speed can create faster timbral movement. Changing curve and noise scale can alter the contour and smoothness of the motion. Raising intensity can expand the range of variation. Lowering gain can make the movement more subtle.

This mode works well for evolving textures, experimental performance, neural resynthesis, and gestural sound design. The user does not need to automate eight latent dimensions manually. The user shapes a visual system, and that system moves through the model.

The central idea is that latent space can be played indirectly. Latent Synth maps visual behavior to neural audio behavior. This makes RAVE's latent space feel less like a hidden technical representation and more like a performable musical material.

The system can be evaluated qualitatively by whether it supports continuity, controllability, exploration, playability, and a clear visual-sonic connection.

X. LIMITATIONS AND FUTURE WORK

Latent Synth currently depends on compatible TorchScript RAVE models. Different exports may have different latent dimensions, frame sizes, channel layouts, or method names. Stronger model validation would make the plugin easier to use.

The flow-field controller currently drives only the first eight latent dimensions. Future versions could expose more dimensions or allow the user to choose which dimensions the flow field controls.

The current flow-to-latent mapping is handcrafted. This makes the system direct and controllable, but it also limits how deeply the mapping can adapt to a specific model or performer. Future versions could learn mappings from gestures, audio examples, or recorded performances.

The plugin could also support deeper DAW integration. MIDI, MPE, host automation, tempo sync, preset management, and modulation routing would make the system more useful in production and performance.

A future version could also include structured user testing with musicians and sound designers. That would help evaluate whether the flow-field interface feels controllable, expressive, and useful in real creative sessions.

XI. CONCLUSION

Latent Synth turns a pre-trained RAVE model into a real-time instrument. It combines a JUCE plugin, TorchScript RAVE inference, circular audio buffering, a p5.js interface, and a flow-field latent controller.

The project’s main contribution is the control layer around the model. RAVE provides the sound engine. Latent Synth gives the user a way to play it.

Native mode supports direct latent editing. Flow-field mode creates continuous visual control. Together, they make RAVE’s latent space easier to explore, shape, and perform inside a DAW.

Latent Synth shows how neural audio models can become interactive instruments when they are paired with strong control design.

REFERENCES

- [1] A. Caillon and P. Esling, “Rave: A variational autoencoder for fast and high-quality neural audio synthesis,” arXiv preprint arXiv:2111.05011, 2021. [Online]. Available: <https://arxiv.org/abs/2111.05011>
- [2] ACIDS IRCAM, “Rave: Realtime audio variational autoencoder,” <https://github.com/acids-ircam/RAVE>, official implementation. Accessed: 2026-05-01.
- [3] Processing Foundation, “p5.js,” <https://p5js.org/>, accessed: 2026-05-01.
- [4] JUCE, “Juce webbrowsercomponent class reference,” https://docs.juce.com/master/classjuce_1_1WebBrowserComponent.html, accessed: 2026-05-01.
- [5] —, “Juce: C++ application framework for audio applications,” <https://juce.com/>, accessed: 2026-05-01.
- [6] Steinberg, “Vst 3 developer portal,” https://steinbergmedia.github.io/vst3_dev_portal/, accessed: 2026-05-01.
- [7] PyTorch, “Torchscript documentation,” <https://docs.pytorch.org/docs/stable/jit.html>, accessed: 2026-05-01.